

[2 일차 오전] Amazon Bedrock 생성형 AI 실습 (Part 1)

여러 **Foundation Model** 을 통합 API 로 활용해, 모델을 직접 학습시키지 않고 **생성형 AI 애플리케이션**을 만드는 **Amazon Bedrock** 을 3 개의 **기업 미니프로젝트**로 실습합니다.

개요

기본 정보

항목	내용
세션명	Amazon Bedrock 생성형 AI (1) — 모델 탐색 · 프롬프트 엔지니어링 · FAQ 챗봇
소요시간	3 시간 (13:30 ~ 16:30) · Lab1·2 각 50 분, Lab3 80 분
실습 도구	SageMaker Studio JupyterLab (Python 3, boto3)
AWS 리전	us-east-1 (버지니아 북부) — Bedrock 모델·기능이 가장 먼저 제공
사용 모델	Claude 3 Haiku anthropic.claude-3-haiku-20240307-v1:0

학습 목표

- Bedrock 클라이언트로 Foundation Model 을 조회하고 **invoke_model** · **converse** API 로 텍스트를 생성한다
- Zero/Few-shot · CoT · System Prompt · **Structured Output(JSON)** 등 프롬프트 기법을 익힌다
- FAQ 검색 + 컨텍스트 주입으로 **멀티턴·스트리밍 챗봇**을 구현한다
- 각 기능이 실제 기업(고객지원 봇·업무 자동화)에서 어떻게 쓰이는지 연결해 설명한다

사전 준비사항

- AWS 계정 로그인 (강사 제공) — **1 일차 오전에 생성한 SageMaker 도메인 사용**
- 실행 역할(IAM)에 AmazonBedrockFullAccess 권한 확인
- Bedrock **모델 액세스(Model access) 활성화** — Claude 모델 사용 허용 (아래 1.4 참고)
- 실습 노트북 3 개: bedrock-p1-lab1~3-student.ipynb

이 강의의 핵심: 3 개 실습 = 기업 미니프로젝트

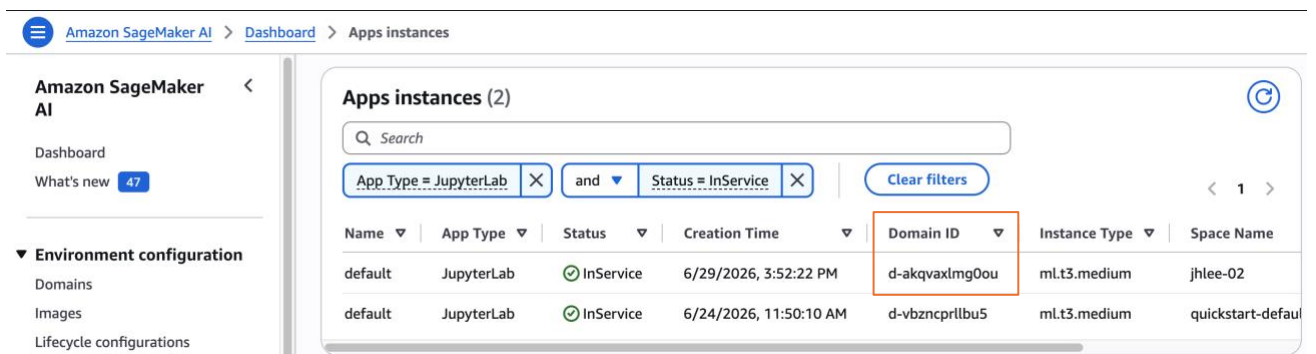
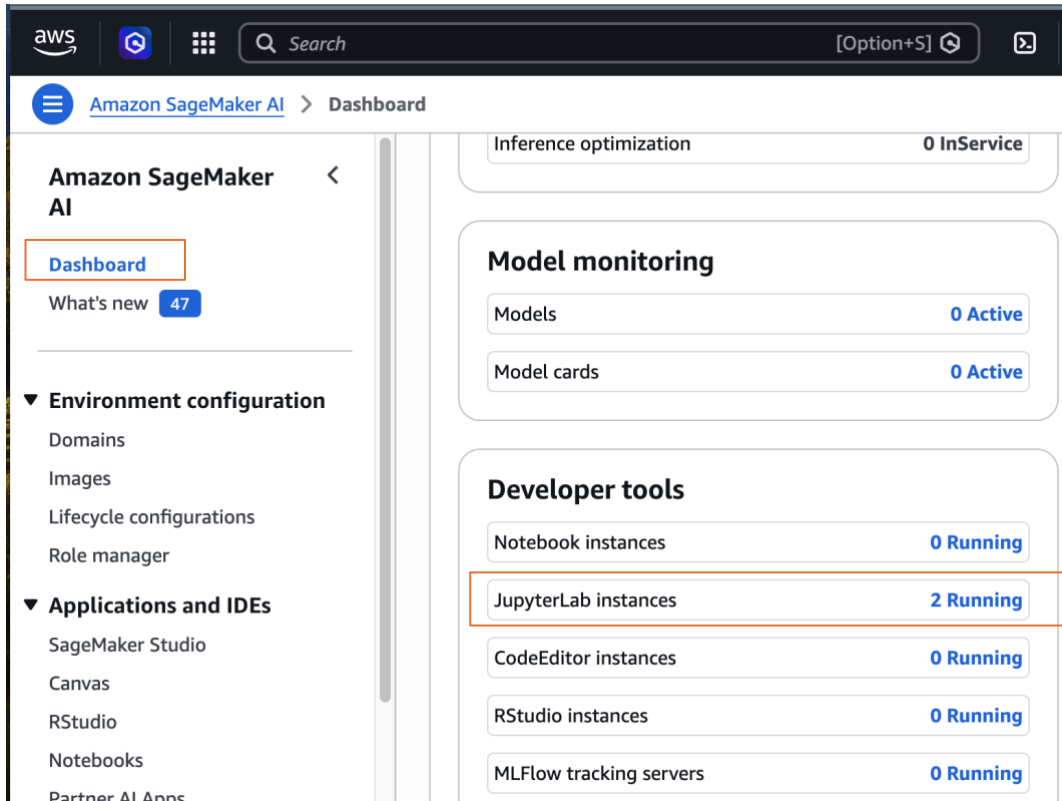
Lab	기업 시나리오	핵심 API / 기법
Lab 1	생성형 AI 도입 TF — 모델 조사·첫 호출·멀티턴 대화	list_foundation_models / invoke_model / converse

Lab	기업 시나리오	핵심 API / 기법
Lab 2	추가 개발 없이 업무 자동화 — 분류·추론·JSON 추출	Zero/Few-shot · CoT · System Prompt · Structured Output
Lab 3	고객지원·사내 헬프데스크 FAQ 챗봇	FAQ 검색 + 컨텍스트 주입 · converse_stream

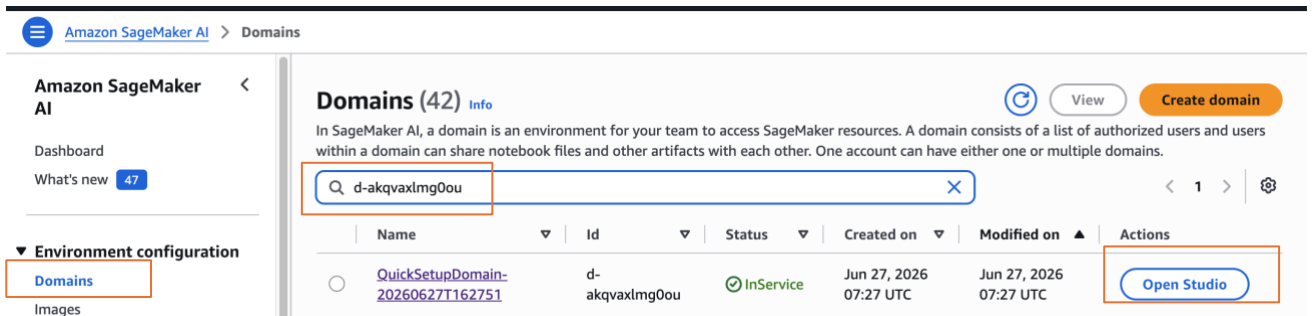
1. 실습 환경 준비

1.1 SageMaker Studio · JupyterLab 열기

1. AWS 콘솔 → SageMaker → 본인 도메인 → Launch → Studio
2. 도메인 찾기



도메인 id 로 검색



3. JupyterLab space 실행 후 Open JupyterLab

1.2 노트북 업로드

- 업로드 아이콘으로 bedrock-p1-lab1~3-student.ipynb 3 개 업로드
- lab1 을 열고 커널이 Python 3 인지 확인

1.3 환경 초기화 실행

첫 코드 셀을 **Shift + Enter** 로 실행합니다. 두 종류의 클라이언트가 생성됩니다.

```
import boto3, json

bedrock = boto3.client('bedrock', region_name='us-east-1') # 모델 목록 조회
bedrock_runtime = boto3.client('bedrock-runtime', region_name='us-east-1') # 텍스트 생성
```

⚠ 자격증명 입력 없음

SageMaker Studio 는 실행 역할(IAM Role)로 자동 인증됩니다. access key 를 코드에 넣지 않습니다.

1.4 Bedrock 모델 액세스 활성화 (중요)

Bedrock 은 처음 사용 시 **모델별 사용 허용을 한 번 신청**해야 합니다. 이게 안 되어 있으면 `AccessDeniedException` 이 납니다.

- 콘솔에서 Amazon Bedrock → Model catalog
- Claude 모델을 선택하여 'Submit use case details' 누르고 입력

The screenshot shows the Amazon Bedrock console interface. On the left, there's a sidebar with navigation links: Amazon Bedrock, Model catalog, and Claude Sonnet 4.5. The main content area displays the details for the Claude Sonnet 4.5 model, including its description and a 'Submit use case details' button. A warning message indicates that Anthropic requires first-time customers to submit use case details before invoking a model.

2. 실습 1 — 생성형 AI 도입 TF: 모델 탐색 & 첫 호출

소요시간 약 50 분 | 난이도 ★★

📋 시나리오 — 생성형 AI 도입 TF

"GPT 같은 걸 우리 업무에 써보자." 모델을 학습시키지 않고 API 호출만으로 시작합니다.

① 모델 조사 → ② 첫 호출(`invoke_model`) → ③ 멀티턴 대화(`converse`)

TODO 1 — Foundation Model 목록 조회

```
response = bedrock.list_foundation_models(
    byOutputModality=_____ # ← 'TEXT'
)
models = response[_____] # ← 'modelSummaries'
```

텍스트 생성 모델 목록과 `modelId` 가 출력됩니다. 이 ID 를 호출에 사용합니다.

TODO 2 — `invoke_model`: 첫 텍스트 생성

`invoke_model` 은 모델별 고유 페이로드 를 씁니다. Claude 는 다음 구조입니다.

```
payload = {
    'anthropic_version': 'bedrock-2023-05-31',
    'max_tokens': _____, # ← 1024
    'messages': [{'role': 'user', 'content': '프롬프트 내용'}]
}
resp = bedrock_runtime.invoke_model(modelId=MODEL_ID, body=json.dumps(payload))
```

TODO 3 — `converse`: 통합 멀티턴 대화

`converse` 는 모델이 달라도 같은 코드 로 쓰는 통합 API 라 실무에서 선호됩니다. 이전 대화를 `messages` 에 누적하면 멀티턴이 됩니다.

💡 실무 연결 — 모델 선택

PoC 는 저렴한 모델로 시작 → 품질 부족 시 상위 모델.

✅ 정답

TODO1: `byOutputModality='TEXT' · response['modelSummaries']`
 TODO2: `max_tokens=1024 · json.loads(resp['body'].read())`
 TODO3: `messages` 리스트에 `{'role':'content': [{'text':...}]}` 누적

3. 실습 2 — 추가 개발 없이 업무 자동화: 프롬프트 엔지니어링

소요시간 약 50 분 | 난이도 ★★☆☆

📖 시나리오 — 데이터 분석가

모델을 학습시키지 않고 '프롬프트만 잘 써서' 업무를 자동화합니다.

분류(Few-shot) · 추론(CoT) · 데이터 추출(JSON) · 톤 제어(System Prompt)

배우는 기법

기법	설명	언제
Zero-shot	예시 없이 지시만으로	간단·명확한 작업
Few-shot	2~5 개 예시를 제공	회사만의 분류 기준 주입
Chain-of-Thought	단계별 추론 유도	복잡한 계산·심사 로직
Structured Output	JSON 형식 강제	시스템에 바로 연결

TODO 1~4 핵심

- **TODO1** Zero vs Few-shot 으로 같은 리뷰 감성 분류 → 예시가 정확도를 높임을 확인
- **TODO2** CoT: [단계 N] 형식으로 단계별 풀이 유도 → 오답률 감소
- **TODO3** System Prompt 로 JSON 만 출력하도록 강제 → json.loads() 로 파싱
- **TODO4** 두 페르소나(예: 친절한 상담원 vs 간결한 전문가) System Prompt 비교

Structured Output: JSON 만 출력하도록 지시

system = "너는 분류기다. 반드시 아래 JSON 만 출력해라: {'유형':..., '긴급도':..., '부서':...}"

answer = claude(user_prompt=문의텍스트, system_prompt=system)

data = json.loads(answer) # 시스템에 바로 연결

💡 실무 연결

고객 문의 → {유형, 긴급도, 부서} 자동 분류로 티켓 시스템에 바로 연결. 모델 학습 0, 프롬프트만으로 자동화.

-- 코드 설명 --

1. 출력 헤더 설정 및 구분선 생성

```
print(f"{'리뷰':<30} {'Zero-shot':>12} {'Few-shot':>10}")
print('-' * 60)
```

- **형식화된 출력 (f-string):** 결과를 표 형태로 깔끔하게 보기 위해 칸을 지정합니다.
 - {'리뷰':<30}: '리뷰'라는 글자를 포함해 총 30 칸을 확보하고 **왼쪽 정렬**합니다.
 - {'Zero-shot':>12}: 총 12 칸을 확보하고 **오른쪽 정렬**합니다.

- {'Few-shot':>10}: 총 10 칸을 확보하고 **오른쪽 정렬**합니다.
- `print('-' * 60)`: 하이픈(-)을 60 번 반복 출력하여 타이틀과 본문 데이터 사이에 구분선을 그려줍니다.

2. 리뷰 목록 반복문 및 API 호출

```
for review in test_reviews:
```

- `test_reviews` 라는 리스트에 담긴 여러 개의 테스트용 상품 리뷰들을 `for` 문을 통해 하나씩 꺼내어 `review` 라는 변수에 할당하며 반복합니다.

① Zero-shot 프롬프트 호출

```
zero_result = claude(
    zero_shot_template.format(review=review),
    temperature=0.1
).strip()
```

- **`zero_shot_template.format(review=review)`**: 사전 예시가 없는 Zero-shot 템플릿 프롬프트에 현재 순서의 `review` 텍스트를 집어넣어 완성합니다.
- **`temperature=0.1`**: 모델의 창의성(무작위성)을 조절하는 파라미터입니다. 감성 분류(긍정/부정 등)처럼 정답이 명확하고 일관된 답변이 필요할 때는 점수를 **0.1**로 낮게 설정하여 매번 똑같이 안정적인 분류 결과를 내도록 유도합니다.
- **`.strip()`**: Claude 모델이 답변 앞뒤로 붙여넣을 수 있는 불필요한 공백이나 줄바꿈(`\n`)을 깔끔하게 제거합니다.

② Few-shot 프롬프트 호출

```
few_result = claude(
    few_shot_template.format(review=review),
    temperature=0.1
).strip()
```

- **`few_shot_template.format(...)`**: "이러한 리뷰는 긍정이다", "이러한 리뷰는 부정이다"와 같이 **몇 개의 예시(Few-shot)**가 미리 작성되어 있는 템플릿에 현재 리뷰를 매칭합니다. 일반적으로 텍스트가 난해하거나 가이드라인이 까다로울 때 Zero-shot 보다 분류 정확도가 더 높습니다.

3. 최종 결과 정렬 출력

```
print(f"{review[:28]:<30} {zero_result:>12} {few_result:>10}")
```

- **review[:28]**: 리뷰 본문이 너무 길면 표의 가로 정렬이 깨질 수 있으므로, 파이썬 슬라이싱을 이용해 **앞글자 28 자까지만** 잘라서 가져옵니다.
- 앞서 지정한 헤더의 간격(:<30, :>12, :>10)과 똑같이 맞추어 출력함으로써, 리뷰 본문과 Zero-shot 결과, Few-shot 결과가 스크립트 창에 바둑판처럼 나란히 정렬되어 출력됩니다.

4. 실습 3 — 고객지원 / 사내 헬프데스크 FAQ 챗봇

소요시간 약 60 분 | 난이도 ★★★★★

시나리오 — 고객지원·사내 헬프데스크 챗봇

FAQ 를 검색해 관련 답변을 모델에 주입(context injection)하고, 멀티턴·스트리밍·품질평가까지 갖춘 실서비스급 챗봇을 만듭니다.

TODO 흐름

8. FAQ 검색 함수 — 질문 키워드로 관련 FAQ 찾기 (`category['keywords']`)
9. FAQChatbot 클래스 — 검색된 FAQ 를 **System Prompt 에 주입**, 대화 히스토리 관리
10. 멀티턴 테스트 — 맥락이 이어지는 대화 확인
11. 스트리밍 — `converse_stream` 으로 실시간 출력
12. 품질 평가 — 테스트 케이스로 정확도 측정

검색 결과를 System Prompt 에 주입 (RAG 의 단순형)

```
context = "\n".join(relevant_faqs)
```

```
system = f"너는 학교 안내 챗봇이다. 아래 FAQ 를 근거로만 답하라.\n{context}"
```

실무 연결 — Part 2 로 가는 다리

이 '검색 후 주입'이 바로 RAG 의 단순형입니다. 키워드 검색은 오타·동의어에 약하므로, Part 2 에서 의미 기반 **벡터 검색(Knowledge Base)**으로 업그레이드합니다.

사용자가 입력한 질문(Query)에서 키워드를 추출하여 가장 연관성이 높은 FAQ(자주 묻는 질문)를 찾아내기 위한 단순 키워드 기반의 검색 시스템을 구현한 코드입니다.

자체적인 점수 산정 알고리즘을 설계하여 점수가 가장 높은 상위 k 개의 질문을 추출해 주는 원리입니다.

1. 함수 정의와 카테고리 가산점 계산

```
def search_faq(query, top_k=3):
    results = []

    for category in FAQ_DATA:
        # 카테고리 키워드가 쿼리에 포함되면 점수 가산
        category_score = sum(
            1 for kw in category['keywords']
            if kw in query
        )
```

- **search_faq(query, top_k=3):** 사용자 질문(query)을 받아 매칭되는 FAQ 를 기본 최대 3 개(top_k=3)까지 찾아주는 함수입니다.
- **category_score (카테고리 매칭):** 전체 FAQ 데이터(FAQ_DATA)를 돌면서, 현재 카테고리의 대표 키워드들이 사용자의 질문 문장(query)에 들어있는지 확인합니다. 포함되어 있다면 개당 **1 점씩**을 부여합니다.
 - 예: 사용자가 "수강신청 취소 방법 알려줘"라고 했다면, '수강신청'이라는 카테고리 키워드가 문장에 포함되므로 카테고리 점수가 올라갑니다.

2. 질문/답변 키워드 매칭 및 가중치 합산

```
for qa in category['qa']:
    # 질문 키워드와 답변 키워드 매칭 점수
    q_score = sum(1 for word in qa['q'].split() if word in query)
    a_score = sum(1 for word in qa['a'].split() if word in query)
    total_score = category_score * 2 + q_score * 3 + a_score
```

- 카테고리 내부의 실제 개별 FAQ 문항(qa)들을 하나씩 분석합니다.
- **q_score (질문 매칭):** FAQ 에 등록된 표준 질문(qa['q'])을 공백 기준(split())으로 쪼개 뒤, 그 단어들이 사용자의 질문(query)에 포함되어 있으면 개당 **1 점씩** 누적합니다.
- **a_score (답변 매칭):** FAQ 에 등록된 답변(qa['a'])을 공백 기준으로 쪼개어 사용자의 질문에 포함되어 있으면 개당 **1 점씩** 누적합니다.
- **total_score (최종 가중치 점수 계산):** 검색의 정확도를 높이기 위해 영역별로 가중치(Weight)를 곱해 합산합니다.

$$\text{total_score} = (\text{category_score} \times 2) + (\text{q_score} \times 3) + \text{a_score}$$

- 즉, 시스템은 질문(q)에 입력된 단어가 일치할 때 가장 높은 가중치(3 배)를 주어 매칭을 시도합니다.

3. 결과 수집 및 필터링

```
if total_score > 0:
    results.append({
        'category': category['category'],
        'question': qa['q'],
        'answer' : qa['a'],
        'score' : total_score
    })
```

- **if total_score > 0::** 질문과 단 한 단어도 겹치지 않아 점수가 0 점인 무관한 FAQ 문항들은 과감히 제외합니다.
- 점수가 1 점이라도 나온 매칭 항목들은 카테고리명, 질문, 답변, 그리고 계산된 최종 점수를 딕셔너리 형태로 리스트(results)에 수집(append)합니다.

4. 내림차순 정렬 및 상위 개수(top_k) 반환

```
# 점수 내림차순 정렬 후 top_k 반환
results.sort(key=lambda x: x['score'], reverse=True)
return results[:top_k]
```

- **results.sort(..., reverse=True)**: 수집된 FAQ 항목들을 score 점수를 기준으로 가장 점수가 높은 순(내림차순)으로 정렬합니다.
- **return results[:top_k]**: 파이썬 슬라이싱을 이용해 가장 연관성이 높은 상위 top_k(여기서는 3 개)만 잘라서 최종 반환합니다.

하단 테스트 결과 해석

코드 맨 밑의 실행 결과를 보면 시스템이 어떻게 작동했는지 직관적으로 알 수 있습니다.

- **사용자 질문**: "수강신청 취소 방법 알려줘"
- **검색 결과**:
 1. [수강신청] 점수:8 Q: 수강신청 취소는 어떻게 하나요?
 - 카테고리 키워드('수강신청')와 질문 내부 단어('수강신청', '취소')가 대거 일치하여 **8 점**이라는 가장 높은 점수로 1 등을 차지했습니다.
 2. [수강신청] 점수:5 Q: 최대 수강학점은 몇 학점인가요?
 - '수강신청' 카테고리 점수와 '수강'이라는 단어가 겹쳐 **5 점**을 받았습니다.
 3. [성적] 점수:5 Q: 재수강 신청 방법은?
 - '신청', '방법' 등의 단어가 겹쳐 공동 **5 점**으로 3 등 안에 진입했습니다.

이처럼 자연어 처리를 복잡하게 쓰지 않고도, 단어의 단순 포함 여부와 가중치 합산만으로 꽤 훌륭한 FAQ 추천 레이어를 만들 수 있음을 보여주는 직관적인 예제 코드입니다!

5. 마무리 & 트러블슈팅

증상	원인 / 해결
AccessDeniedException	Bedrock Model access 미승인 → 콘솔에서 Claude 모델 허용 (1.4)
ValidationException	invoke_model 페이로드 구조 오류 → anthropic_version·max_tokens·messages 확인
한국어가 어색	max_tokens 가 너무 작으면 잘림 → 1024 이상 권장
응답이 매번 다름	temperature 가 높음 → 분류·추출은 0~0.2 로 낮추기